УНИВЕРСИТЕТ ИТМО

Домашнее задание № 3
Свой MapReduce и DuckDB поверх объектного хранилища

Вариант 9. Программист, разработчик на мобильных платформах

Выполнил: Вяхирев Иван · ИСУ 465468 · группа J3310

Исходный код всех программ лежит в репозитории EngData_2026_3 на GitHub. Все запуски сделаны на одном компьютере:

```
Python: Python 3.11.3
CPU: AMD Ryzen 7 2700X Eight-Core Processor
RAM: 31,9 GB
Logical processors: 16
```

1. Развёртывание RustFS

RustFS поднят так же, как в ДЗ 2, но на новых пустых папках `C:\rustfs3\disk1…disk4`.

```
docker run -d --name rustfs --network lab --user root -p 9000:9000 -p 9001:9001 `
  -v C:\rustfs3\disk1:/data/rustfs0 -v C:\rustfs3\disk2:/data/rustfs1 `
  -v C:\rustfs3\disk3:/data/rustfs2 -v C:\rustfs3\disk4:/data/rustfs3 `
  -e "RUSTFS_VOLUMES=/data/rustfs{0...3}" -e RUSTFS_CONSOLE_ENABLE=true `
  -e RUSTFS_ACCESS_KEY=rustfsadmin -e RUSTFS_SECRET_KEY=rustfsadmin `
  -e RUSTFS_UNSAFE_BYPASS_DISK_CHECK=true rustfs/rustfs:latest
docker ps
```

```
PS D:\!Code\папкипитона\!!!нейронки\itmo_labs\3 курс\EngData_2026_3> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED             STATUS
PORTS                                                                                                 NAMES
8735fc53b548   rustfs/rustfs:latest                 "/entrypoint.sh rust…"   About an hour ago   Up About an hour
0.0.0.0:9000-9001->9000-9001/tcp                                                                      rustfs
e39e1127044b   agent-for-service-anomalies-app      "start"                  2 weeks ago         Up 2 hours (healthy)
0.0.0.0:8000->8000/tcp                                                                                observer-app
db1238617db8   agent-for-service-anomalies-mlflow   "mlflow server --hos…"   2 weeks ago         Up 2 hours (healthy)
0.0.0.0:5000->5000/tcp                                                                                observer-mlflow
1f690831fb20   chrislusf/seaweedfs:4.15             "/entrypoint.sh serv…"   2 weeks ago         Up 2 hours (healthy)
7333/tcp, 8080/tcp, 8333/tcp, 9333/tcp, 18080/tcp, 18888/tcp, 19333/tcp, 0.0.0.0:8888->8888/tcp      observer-seaweedfs
bcf7d9ce7c7f   postgres:16-alpine                   "docker-entrypoint.s…"   2 weeks ago         Up 2 hours (healthy)
0.0.0.0:5433->5432/tcp                                                                                observer-postgres
PS D:\!Code\папкипитона\!!!нейронки\itmo_labs\3 курс\EngData_2026_3>
```

Контейнер rustfs запущен

2. Загрузка данных

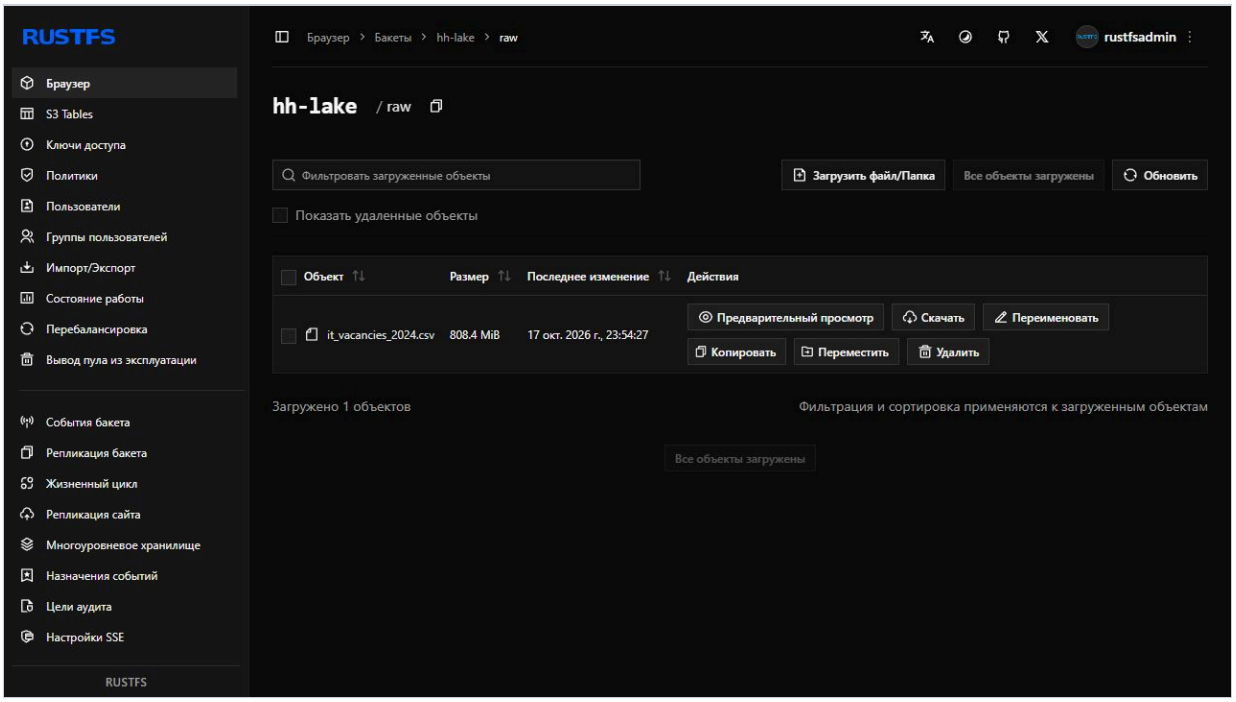
Создан бакет `hh-lake`, в него через AWS CLI загружен CSV с вакансиями.

```
aws s3 mb s3://hh-lake
aws s3 cp /data/it_vacancies_2024.csv s3://hh-lake/raw/it_vacancies_2024.csv
aws s3 ls s3://hh-lake/ --recursive --human-readable --summarize
```

```
PS D:\!Code\папкипитона\!!!нейронки\itmo_labs\3 курс\EngData_2026_3> aws s3 ls s3://hh-lake/ --recursive --human-readable --summarize
2026-10-17 20:54:27  808.4 MiB raw/it_vacancies_2024.csv
2026-10-18 09:50:26    6.0 MiB raw/it_vacancies_2024.parquet
2026-10-18 09:53:11   20.9 KiB results/duckdb/k1.csv
2026-10-18 09:53:11  332.6 KiB results/duckdb/k2.csv
2026-10-18 09:53:12    2.1 MiB results/duckdb/k3.csv
2026-10-18 09:55:08   20.9 KiB results/duckdb_parquet/k1.csv
2026-10-18 09:55:09  332.6 KiB results/duckdb_parquet/k2.csv
2026-10-18 09:55:09    2.1 MiB results/duckdb_parquet/k3.csv
2026-10-18 09:52:08   20.9 KiB results/mapreduce/k1.csv
2026-10-18 09:52:09  332.3 KiB results/mapreduce/k2.csv
2026-10-18 09:52:09    2.1 MiB results/mapreduce/k3.csv
2026-10-18 09:55:01   20.9 KiB results/mapreduce_base/k1.csv
2026-10-18 09:55:01  332.3 KiB results/mapreduce_base/k2.csv
2026-10-18 09:55:02    2.1 MiB results/mapreduce_base/k3.csv

Total Objects: 14
   Total Size: 824.3 MiB
```

Содержимое бакета hh-lake: исходный CSV, Parquet для бонуса и все результаты



Объект raw/it_vacancies_2024.csv в бакете hh-lake, 808,4 МиБ

Условие варианта

Роль в выгрузке у всех разработчиков общая, «Программист, разработчик». Мобильных выделяю по названию вакансии в нижнем регистре: в нём есть ios, android, flutter, react native, kmp, multiplatform, «мобильн» или mobile. Тестировщиков с qa, aqa, «тестиров» и test в названии не беру. В DuckDB условие записано так:

```
WHERE role = 'Программист, разработчик'
  AND regexp_matches(lower(name), '\bios\b|android|flutter|react[ -]?native|\bkmp\b|multiplatform|
мобильн|\bmobile\b')
  AND NOT regexp_matches(lower(name), '\bqa\b|\baqa\b|тестиров|\btest')
```

В Python то же условие проверяется функцией `is_variant` в `common.py`. Из 229 388 строк в вариант попало 3 704 вакансии, из них 2 739 с навыками и 1 229 с зарплатой в рублях.

3. Подсчёт навыков своим MapReduce

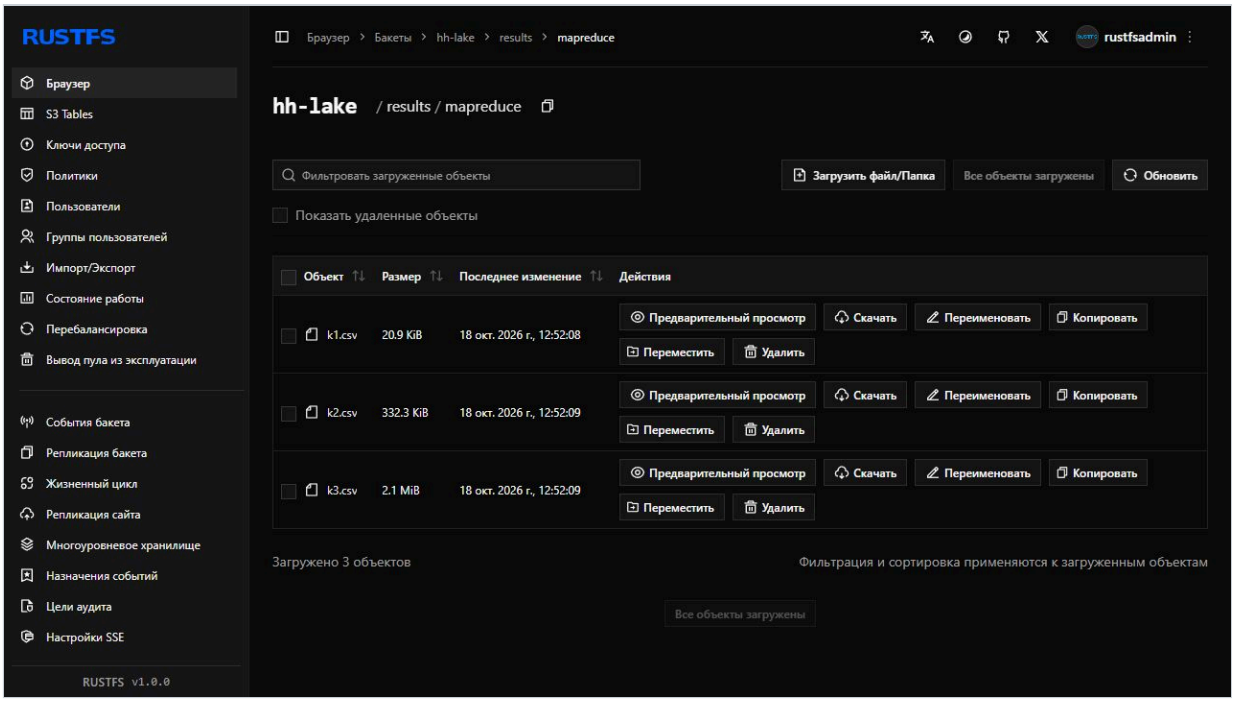
Программа `mr_skills.py` читает объект из RustFS кусками по 8 МБ запросами с заголовком Range, 102 куска. Строки, дочитанные в одном куске, становятся одной map-задачей. Задачи выполняют 4 процесса.

- **map** пропускает строки не своего варианта. Для вакансии варианта берёт навыки без повторов, сортирует и выдаёт все сочетания по 1, 2 и 3 навыка со значением 1, а ещё пару (число навыков, 1), из неё считается число вакансий.
- **Combiner** сразу складывает одинаковые ключи внутри маппера. **reduce** суммирует.
- **Shuffle.** Ключ уходит в reducer номер crc32(ключ) mod 4. Встроенный hash в Python у каждого процесса свой, поэтому один и тот же ключ мог бы попасть в разные reducer'ы.

```
MapReduce: workers=4, reducers=4, chunk=8 МБ, combiner=да
Объект s3://hh-lake/raw/it_vacancies_2024.csv: 847619384 байт, читаем кусками по 8 МБ, кусков: 102
map-задач: 102, пар (ключ, значение) из map: 194277, ушло в shuffle: 158245, перезапусков: 0
Прочитано строк CSV: 229388
Вакансий варианта: 3704, из них с навыками: 2739

Рост числа сочетаний: вакансия с n навыками даёт C(n,1)=n навыков, C(n,2) пар и C(n,3) троек
 k     разных  всего вхождений
 1       1149            14977   -> s3://hh-lake/results/mapreduce/k1.csv
 2      13049            47503   -> s3://hh-lake/results/mapreduce/k2.csv
 3      63970           128093   -> s3://hh-lake/results/mapreduce/k3.csv
Навыков в вакансии: в среднем 5.5, максимум 30
```

Результаты для навыков, пар и троек записаны в хранилище, файлы `k1.csv`, `k2.csv`, `k3.csv`, они видны и в списке объектов в разделе 2. Файлы результатов перезаписываются при каждом запуске, поэтому время изменения на разных снимках разное.



Папка results/mapreduce в бакете hh-lake

Почему пар и троек больше, чем навыков

Вакансия с n навыками даёт n навыков, C(n,2) пар и C(n,3) троек:

$$C(n,2) = n(n-1)/2, \quad C(n,3) = n(n-1)(n-2)/6$$

В среднем в вакансии 5,5 навыка, максимум 30. При n = 5 это 5 навыков, 10 пар и 10 троек, а при n = 30 уже 435 пар и 4 060 троек. Поэтому всего вхождений 14 977, 47 503 и 128 093, а разных сочетаний 1 149, 13 049 и 63 970.

```
Топ-10, навыки:
    1155  Android
    1039  Kotlin
     882  iOS
     822  Git
     682  Swift
     605  Android SDK
     590  Java
     391  Flutter
     330  MVVM
     307  ООП

Топ-10, пары:
     760  Android + Kotlin
     538  Swift + iOS
     503  Android SDK + Kotlin
     484  Java + Kotlin
     454  Android + Android SDK
     410  Android + Java
     368  Android + Git
     330  Android SDK + Java
     294  Git + Kotlin
     280  Git + iOS

Топ-10, тройки:
     385  Android + Android SDK + Kotlin
     355  Android + Java + Kotlin
     282  Android SDK + Java + Kotlin
     250  Android + Android SDK + Java
     225  Android + Git + Kotlin
     177  Android + Android SDK + Git
     177  Android SDK + Git + Kotlin
     160  Git + Java + Kotlin
     152  Git + Swift + iOS
     151  Android + Kotlin + MVVM
```

4. Подсчёт в DuckDB и сверка

`duck_skills.py` читает тот же объект из RustFS через расширение httpfs. Навыки разбиваются функцией `string_split`, пары и тройки получаются соединением таблицы навыков с собой по номеру вакансии с условием a.skill < b.skill < c.skill.

```
SELECT a.skill AS skill1, b.skill AS skill2, c.skill AS skill3, count(*) AS count
FROM mobile_skill a
JOIN mobile_skill b ON a.id = b.id AND a.skill < b.skill
JOIN mobile_skill c ON b.id = c.id AND b.skill < c.skill
GROUP BY ALL
```

```
DuckDB (csv): вакансий варианта: 3704, из них с навыками: 2739
 k     разных  всего вхождений
 1       1149            14977   -> s3://hh-lake/results/duckdb/k1.csv
 2      13049            47503   -> s3://hh-lake/results/duckdb/k2.csv
 3      63970           128093   -> s3://hh-lake/results/duckdb/k3.csv
```

`verify.py` сравнивает файлы обоих способов построчно, в обе стороны через EXCEPT:

```
k=1: mapreduce vs duckdb: MapReduce строк 1149, сумма 14977 | DuckDB строк 1149, сумма 14977 | есть только
в MR: 0, только в DuckDB: 0 -> СОВПАЛО
k=2: mapreduce vs duckdb: MapReduce строк 13049, сумма 47503 | DuckDB строк 13049, сумма 47503 | есть
только в MR: 0, только в DuckDB: 0 -> СОВПАЛО
k=3: mapreduce vs duckdb: MapReduce строк 63970, сумма 128093 | DuckDB строк 63970, сумма 128093 | есть
только в MR: 0, только в DuckDB: 0 -> СОВПАЛО

ИТОГ: результаты полностью совпадают
```

```
PS D:\!Code\папкипитона\!!!нейронки\itmo_labs\3 курс\EngData_2026_3> python verify.py  | Tee-Object logs\04_verify.txt
k=1: mapreduce vs duckdb: MapReduce строк 1149, сумма 14977 | DuckDB строк 1149, сумма 14977 | есть только в MR: 0, только в DuckDB: 0 -> СОВПАЛО
k=2: mapreduce vs duckdb: MapReduce строк 13049, сумма 47503 | DuckDB строк 13049, сумма 47503 | есть только в MR: 0, только в DuckDB: 0 -> СОВПАЛО
k=3: mapreduce vs duckdb: MapReduce строк 63970, сумма 128093 | DuckDB строк 63970, сумма 128093 | есть только в MR: 0, только в DuckDB: 0 -> СОВПАЛО

ИТОГ: результаты полностью совпадают
```

Повторный запуск verify.py

5. Отказ маппера

С ключом `--fail-map 3` map-задача 3 падает на первой попытке. Мастер ловит ошибку и запускает её заново на тех же данных:

```
MapReduce: workers=4, reducers=4, chunk=8 МБ, combiner=да
Объект s3://hh-lake/raw/it_vacancies_2024.csv: 847619384 байт, читаем кусками по 8 МБ, кусков: 102
map-задача 3, попытка 1: ОШИБКА RuntimeError('имитация отказа маппера'), перезапускаю
map-задача 3 выполнена с попытки 2
map-задач: 102, пар (ключ, значение) из map: 194277, ушло в shuffle: 158245, перезапусков: 1
Прочитано строк CSV: 229388
Вакансий варианта: 3704, из них с навыками: 2739
```

```
k=3: mapreduce vs duckdb: MapReduce строк 63970, сумма 128093 | DuckDB строк 63970, сумма 128093 | есть
только в MR: 0, только в DuckDB: 0 -> СОВПАЛО

ИТОГ: результаты полностью совпадают
```

Повтор возможен, потому что map не меняет ничего снаружи: на тех же входных данных он даёт тот же результат.

6. Профилирование

`bench.py` запускает каждый способ 3 раза отдельным процессом. Время считается от начала до конца программы, пик памяти это максимум суммарной памяти процесса и всех его дочерних процессов, замер каждые 20 мс.

```
MapReduce (4 процесса, combiner) | запуск 1: 37.51 с, пик 486.9 МБ
MapReduce (4 процесса, combiner) | запуск 2: 38.27 с, пик 487.1 МБ
MapReduce (4 процесса, combiner) | запуск 3: 33.76 с, пик 479.4 МБ
DuckDB, CSV из S3 | запуск 1: 20.85 с, пик 187.8 МБ
DuckDB, CSV из S3 | запуск 2: 20.08 с, пик 190.3 МБ
DuckDB, CSV из S3 | запуск 3: 19.92 с, пик 190.4 МБ
```

вариант	время, с, среднее	мин–макс	пик памяти, МБ, среднее	мин–макс
MapReduce (4 процесса, combiner)	36.51	33.76–38.27	484	479.4–487.1
DuckDB, CSV из S3	20.28	19.92–20.85	190	187.8–190.4

- **Время.** DuckDB быстрее примерно в 1,8 раза. Почти всё время MapReduce уходит на чтение 848 МБ и разбор CSV в одном главном процессе на Python. Мапперам достаются только уже разобранные строки, а работы у них мало: в вариант попадает 3 704 строки из 229 388. DuckDB разбирает CSV своим кодом на C++ в несколько потоков.
- **Память.** У MapReduce пик около 484 МБ, в 2,5 раза больше, чем у DuckDB с его 190 МБ. Это главный процесс и 4 процесса-маппера, у каждого свой интерпретатор Python, а все куски файла сразу отправляются в очередь к мапперам. DuckDB работает в одном процессе и читает CSV потоком, не держа файл целиком.

7. Влияние навыков на зарплату

Зарплата вакансии это середина вилки, а если указана одна граница, то она. Берутся только рубли, это 1 229 вакансий, медиана по варианту 170 000 ₽. Прибавка пары считается от более частого навыка пары, частота взята по всем вакансиям варианта: медиана пары минус медиана базового навыка. Прибавка тройки считается от самой частой пары внутри неё. Чтобы медиана не строилась по паре вакансий, взяты навыки, которые встречаются в 30 вакансиях с зарплатой и больше, пары в 20 и больше, тройки в 15 и больше.

Навыки выше медианы варианта

навык	вакансий с з/п	медиана, ₽	разница с 170 000
Dagger 2	37	200 000	30 000
Kotlin	330	200 000	30 000
SOLID	69	200 000	30 000
RxJava	43	180 000	10 000
Android SDK	206	175 000	5 000
Retrofit	43	175 000	5 000
Swift	192	175 000	5 000

Навыки ниже медианы варианта

навык	вакансий с з/п	медиана, ₽	разница с 170 000
JSON API	37	110 000	-60 000
MVC	31	110 000	-60 000
HTML	32	125 000	-45 000
JavaScript	91	125 000	-45 000
Xcode	43	125 000	-45 000
SQL	43	130 000	-40 000
Dart	134	135 000	-35 000
REST API	72	135 000	-35 000
Flutter	196	140 000	-30 000
ООП	138	140 000	-30 000

Пары, которые сильнее всего повышают медиану

базовый навык	частота	медиана базового	добавленный	вакансий с з/п	медиана пары	прибавка
iOS	882	150 000	SOLID	33	210 000	60 000
ООП	307	140 000	SOLID	26	200 000	60 000
Kotlin	1039	200 000	MVI	20	245 000	45 000
Android	1155	160 000	Kotlin	251	200 000	40 000
Android	1155	160 000	Dagger 2	36	200 000	40 000
Swift	682	175 000	SOLID	21	215 000	40 000
JavaScript	158	125 000	React	30	160 000	35 000
Git	822	150 000	SOLID	40	175 000	25 000
Git	822	150 000	PostgreSQL	20	175 000	25 000
iOS	882	150 000	Swift	147	175 000	25 000

Пары, которые сильнее всего понижают медиану

базовый навык	частота	медиана базового	добавленный	вакансий с з/п	медиана пары	прибавка
Kotlin	1039	200 000	Android Studio	20	105 000	-95 000
Kotlin	1039	200 000	MVP	24	110 000	-90 000
Kotlin	1039	200 000	ООП	48	130 000	-70 000
iOS	882	150 000	JSON API	24	80 000	-70 000
Git	822	150 000	Objective-C	23	95 000	-55 000
Android	1155	160 000	JSON API	25	110 000	-50 000
Android SDK	605	175 000	ООП	38	125 000	-50 000
Git	822	150 000	HTML	21	100 000	-50 000
Kotlin	1039	200 000	MVVM	75	150 000	-50 000
Kotlin	1039	200 000	Git	123	150 000	-50 000

Тройки, которые сильнее всего повышают медиану

базовая пара	частота	медиана пары	добавленный	вакансий с з/п	медиана тройки	прибавка
Android + MVVM	166	150 000	MVI	15	250 000	100 000
Java + Kotlin	484	160 000	Dagger 2	17	260 000	100 000
Kotlin + MVVM	199	150 000	MVI	16	247 500	97 500
Swift + iOS	538	175 000	SOLID	17	250 000	75 000
Android + Android SDK	454	180 000	Hilt	15	250 000	70 000
Android SDK + Kotlin	503	180 000	Hilt	15	250 000	70 000
Android + Android SDK	454	180 000	MVI	15	245 000	65 000
Android SDK + Kotlin	503	180 000	MVI	16	245 000	65 000
Git + JavaScript	74	115 000	React	19	170 000	55 000
iOS + ООП	112	160 000	SOLID	15	215 000	55 000

Тройки, которые сильнее всего понижают медиану

базовая пара	частота	медиана пары	добавленный	вакансий с з/п	медиана тройки	прибавка
Android + Kotlin	760	200 000	MVP	17	110 000	-90 000
Android + Android SDK	454	180 000	MVP	15	110 000	-70 000
Android SDK + Kotlin	503	180 000	MVP	17	110 000	-70 000
Swift + iOS	538	175 000	MVC	15	110 000	-65 000
Android + Kotlin	760	200 000	ООП	40	140 000	-60 000
Android + Java	410	170 000	iOS	15	115 000	-55 000
Git + Swift	183	150 000	Objective-C	20	95 000	-55 000
React + React Native	95	170 000	iOS	20	115 000	-55 000
Swift + iOS	538	175 000	Android	19	120 000	-55 000
Git + iOS	280	149 500	Objective-C	22	95 000	-54 500

Почему так

- **Повышают** навыки современной нативной разработки под Android: MVI, Hilt, Dagger 2, Kotlin вместе с Android, и SOLID у iOS и Swift. Такие слова пишут в вакансиях middle и senior, отсюда и зарплата.
- **Понижают** старые подходы и признаки младших позиций. MVP и MVC это архитектуры, на смену которым пришли MVVM и MVI. Objective-C это поддержка старого iOS-кода. ООП, HTML, JSON API и Android Studio как отдельный навык пишут в вакансиях для начинающих. JavaScript, Dart и Flutter дешевле нативной разработки: это кроссплатформенные вакансии, часто в небольших компаниях.

- Это сравнение медиан, а не доказательство, что навык сам поднимает зарплату. Многие пары и тройки собраны всего по 15–40 вакансиям с зарплатой, поэтому разница в 20–30 тысяч может быть случайной. Устойчивее всего вывод про MVP, Objective-C и ООП: они попадают в понижающие и у пар, и у троек.

8. Региональный анализ

Взяты три региона, где больше всего вакансий варианта: Москва, Санкт-Петербург, Новосибирск.

Таблица 1. Зарплата в рублях

регион	вакансий	с зарплатой	медиана, ₽
Москва	1731	511	206 000
Санкт-Петербург	575	175	160 000
Новосибирск	135	43	150 000

Таблица 2. Самые частые навыки, пары и тройки: сочетание · доля вакансий с навыками · медиана зарплаты

	Москва	Санкт-Петербург	Новосибирск
навык	Android · 39.1 % · 200 000 ₽	Android · 42.7 % · 160 000 ₽	Android · 36.3 % · 180 000 ₽
	Kotlin · 38.3 % · 250 000 ₽	Kotlin · 41.7 % · 200 000 ₽	iOS · 35.4 % · 162 500 ₽
	iOS · 30.2 % · 190 000 ₽	iOS · 35.8 % · 160 000 ₽	Kotlin · 32.7 % · 190 000 ₽
	Git · 26.7 % · 190 000 ₽	Swift · 26.4 % · 200 000 ₽	Git · 31.9 % · 137 500 ₽
	Swift · 24.6 % · 210 500 ₽	Java · 23.2 % · 160 000 ₽	Swift · 27.4 % · 122 500 ₽
пара	Android + Kotlin · 27 % · 247 500 ₽	Android + Kotlin · 31.9 % · 185 000 ₽	Swift + iOS · 19.5 % · 122 500 ₽
	Swift + iOS · 18.9 % · 206 000 ₽	Swift + iOS · 21.7 % · 200 000 ₽	Android + Kotlin · 17.7 % · 200 000 ₽
	Android SDK + Kotlin · 17.1 % · 250 000 ₽	Java + Kotlin · 21.2 % · 160 000 ₽	Java + Kotlin · 16.8 % · 190 000 ₽
тройка	Android + Android SDK + Kotlin · 12.8 % · 250 000 ₽	Android + Java + Kotlin · 16.5 % · 160 000 ₽	Android + Android SDK + Kotlin · 8.8 % · 200 000 ₽
	Android + Java + Kotlin · 11.5 % · 230 000 ₽	Android + Android SDK + Kotlin · 12.1 % · 220 000 ₽	Android + Git + Kotlin · 8.8 % · 175 000 ₽
	Android SDK + Java + Kotlin · 9.3 % · 230 000 ₽	Android SDK + Java + Kotlin · 8.6 % · 150 000 ₽	Git + Kotlin + SOLID · 8.8 % · 175 000 ₽

Наблюдения

- Москва это почти половина всех мобильных вакансий, 1 731 из 3 704. Медиана 206 000 ₽ на 46 000 выше петербургской и на 56 000 выше новосибирской.
- В Москве и Петербурге основа это нативный Android: пара Android и Kotlin самая частая. Но в Москве её медиана 247 500 ₽, а в Петербурге 185 000 ₽. В Петербурге в первую пятёрку навыков входит Java, 23,2 %, а пара Java и Kotlin с медианой 160 000 ₽ стоит на третьем месте: это старые Android-проекты, которые переводят с Java на Kotlin.
- В Новосибирске iOS почти догнал Android, 35,4 % против 36,3 %, и на первом месте среди пар Swift и iOS. При этом медиана этой пары там 122 500 ₽, а в Москве 206 000 ₽. Разница в зарплате за один и тот же стек больше всего именно у iOS.
- В Новосибирске зарплата указана только в 43 вакансиях, а тройки собраны по 10 вакансий, поэтому его медианы самые неточные.

9. Бонус: оптимизация

Для DuckDB CSV один раз переложен в Parquet в том же бакете, без колонки description и с сжатием zstd. Описание вакансии занимает почти весь файл, а для подсчёта навыков оно не нужно.

```
Готово: s3://hh-lake/raw/it_vacancies_2024.parquet, строк: 229388

Время: 18.91 с, пик памяти: 339.3 МБ
```

Parquet занял 6,0 МиБ вместо 808,4 МиБ CSV. Для MapReduce «до» это один процесс без combiner, «после» это 4 процесса с combiner. Каждый вариант запущен 3 раза:

вариант	время, с, среднее	мин–макс	пик памяти, МБ, среднее	мин–макс
MapReduce (4 процесса, combiner)	33.08	32.81–33.28	483	482.8–483.8
DuckDB, CSV из S3	20.29	20.04–20.46	192	188.4–196.5
ДО: MapReduce, 1 процесс, без combiner	35.67	33.73–37.28	343	340.7–344.3
ПОСЛЕ: DuckDB, Parquet из S3	1.90	1.81–2.01	107	105.8–107.6

```
k=1: mapreduce_base vs duckdb_parquet: MapReduce строк 1149, сумма 14977 | DuckDB строк 1149, сумма 14977 |
есть только в MR: 0, только в DuckDB: 0 -> СОВПАЛО
k=2: mapreduce_base vs duckdb_parquet: MapReduce строк 13049, сумма 47503 | DuckDB строк 13049, сумма 47503
| есть только в MR: 0, только в DuckDB: 0 -> СОВПАЛО
k=3: mapreduce_base vs duckdb_parquet: MapReduce строк 63970, сумма 128093 | DuckDB строк 63970, сумма
128093 | есть только в MR: 0, только в DuckDB: 0 -> СОВПАЛО

ИТОГ: результаты полностью совпадают
```

- **DuckDB:** 20,29 с и 192 МБ до, 1,90 с и 107 МБ после. Быстрее в 10,7 раза, памяти почти вдвое меньше, результат тот же.
- **MapReduce:** combiner сократил число пар, которые уходят в shuffle, со 194 277 до 158 245, на 19 %. Время с combiner и 4 процессами 33,08 с против 35,67 с до, быстрее всего на 7 %, а памяти стало больше, 483 МБ против 343 МБ. Причина та же, что в разделе 6: время уходит на разбор CSV в главном процессе, а не на map и shuffle. Чтобы заметно ускорить свой MapReduce, надо разбирать CSV параллельно или читать тот же Parquet.